CHAPTER 2

Machine Learning from First Principles

In this chapter

Introduction: The Model That Learned the Wrong Thing . . . 2

2.1 Core Concepts: The Vocabulary of Learning . . . 3

2.2 The Four Paradigms of Learning . . . 6

2.3 The Training Loop . . . 8

2.4 Generalisation: The Whole Game . . . 9

2.5 Splitting Data, and the Four Ways to Cheat Yourself . . . 10

2.6 Where This Sits in a Claude-Powered System . . . 11

2.7 Code Walkthrough: A Classifier From Scratch . . . 12

2.8 Hands-on Labs . . . 15

2.9 Lab Solutions . . . 15

2.10 Best Practices . . . 16

2.11 Common Mistakes . . . 17

2.12 Performance Considerations . . . 18

2.13 Security Considerations . . . 18

2.14 Real Production Examples . . . 19

2.15 Mini Project: The Hybrid Router . . . 19

2.16 Certification Notes . . . 22

2.17 Practice Questions . . . 22

2.18 Answers and Explanations . . . 24

2.19 Ten Practical Exercises . . . 25

2.20 Five Challenge Labs . . . 26

2.21 Interview Questions . . . 26

2.22 Summary . . . 26

2.23 Glossary . . . 27

2.24 Further Reading . . . 28

NOTE

Learning Objectives

By the end of this chapter you will be able to:

1. Define *dataset*, *feature*, *label*, *model*, *loss*, *optimisation* and *generalisation*, and use them precisely.
2. Distinguish **supervised**, **unsupervised**, **reinforcement** and **self-supervised** learning, and identify which one any given system uses.
3. Describe the training loop step by step and explain what changes on each iteration.
4. Explain what a loss function is and how gradient descent minimises it — without calculus.
5. Diagnose **underfitting** and **overfitting** from evidence, and explain the bias–variance trade-off in your own words.
6. Design a correct train / validation / test split and detect the four common forms of data leakage.
7. Explain why **self-supervised learning** was the unlock that made large language models possible.
8. Implement a working classifier from scratch in plain Python, then compare it against a Claude API call on cost, latency, accuracy and maintainability, and defend the choice.

NOTE

Prerequisites

Chapter 1. You should be comfortable with the nesting of AI, machine learning, deep learning, generative AI and LLMs; with the distinction between training and inference; and with the idea that the model is a component rather than the system.

No mathematics beyond arithmetic is assumed. Where the underlying idea is calculus, this chapter gives you the intuition and tells you honestly that it is doing so.

Introduction: The Model That Learned the Wrong Thing

A hospital group once built a model to predict which pneumonia patients were at high risk of dying, so that low-risk patients could safely be sent home. The model performed well on every test they ran. Then a researcher inspected what it had actually learned, and found a rule that read, in effect: *patients who also have asthma are at low risk*.

This is the opposite of the truth. Asthma makes pneumonia considerably more dangerous. But in the training data, asthmatic pneumonia patients had unusually good outcomes — because clinicians knew they were high-risk and admitted them straight to intensive care. The model had not learned about biology. It had learned about hospital triage policy, and it had learned it perfectly.

Nothing malfunctioned. The algorithm found a genuine statistical regularity in the data it was given. It simply found the wrong one, and had it been deployed it would have sent exactly the wrong patients home.

This chapter exists because that failure mode never goes away. It is present in a spam filter, in a fraud model, in a recommendation engine, and — in a subtler and more interesting form — in Claude. Every machine-learning system learns *whatever pattern in the data best reduces its error*, which is very often not the pattern you had in mind. Understanding how learning actually works is what lets you predict

where that gap will open.

There is also a direct payoff for the certification. Domain 4 asks you to design evaluations for prompts; Domain 5 asks you to reason about degradation over long context; Domain 1 asks when a simpler system beats an agent. Every one of those questions is a machine-learning question wearing different clothes. Answering them well requires the vocabulary this chapter builds.

2.1 Core Concepts: The Vocabulary of Learning

2.1.1 The dataset, and what it is made of

A **dataset** is a collection of **examples**. An example is one thing you want the system to reason about: one email, one photograph, one customer, one sentence.

Each example is described by **features** — the measurable properties you feed to the model. For an email, features might be the number of words, whether the sender is in your address book, how many links it contains. For a photograph, the features are the pixel values.

In **supervised** learning, each example also carries a **label**: the correct answer. Spam or not spam. Malignant or benign. The price the house actually sold for.

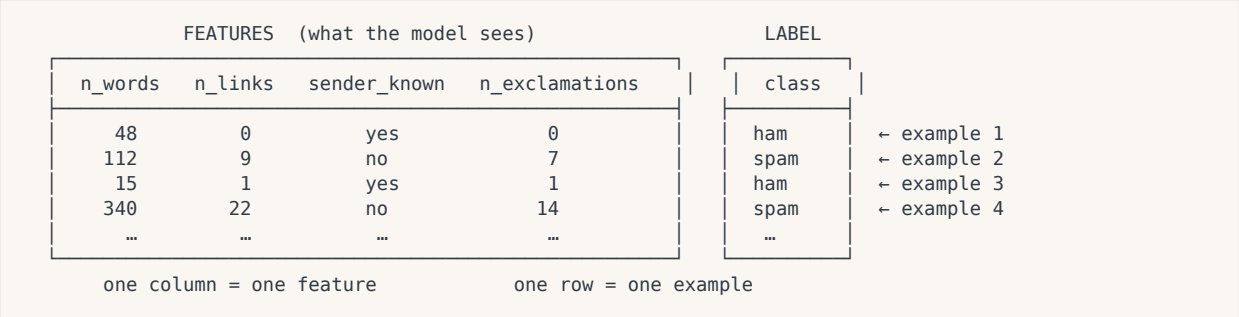


Figure 2.1 — The anatomy of a supervised dataset.

> DEFINITION
>
> **Dataset, example, feature, label**
>
> A **dataset** is a set of **examples** (rows). Each example is described by **features** (columns) and, in supervised learning, carries a **label** (the correct output). The goal of training is to find a function that maps features to labels well enough that it also works on examples the system has never seen.

2.1.2 The model, and what "learning" changes

A **model** is a function with adjustable numbers in it. Give it features, it returns a prediction. The adjustable numbers are the **parameters** you met in Chapter 1.

Here is the smallest honest example. Suppose we predict whether an email is spam using a single number, a *score*:

```
score = w1 x n_links  +  w2 x n_exclamations  +  w3 x (sender_known ? 0 : 1)  +  b
if score > 0  →  predict SPAM
```

```
else          →  predict HAM
```

`w1`, `w2`, `w3` and `b` are the parameters. Before training they are arbitrary — often small random values, or zero. **Learning is nothing more than the process of choosing better values for them.** That is the whole of it. A model with seventy billion parameters differs from this one in scale and in structure, not in kind.

VISUAL ANALOGY

The mixing desk

Picture a sound engineer at a mixing desk with four sliders. Each slider controls how much one instrument contributes to the final mix. The engineer plays a track, listens, decides it is too heavy on the bass, nudges that slider down, and listens again. Repeat a few thousand times and the mix is good.

The sliders are the parameters. "Listening and judging" is the **loss function** — a number saying how bad the current mix is. "Nudging the sliders in the direction that improves it" is **gradient descent**. Training is that loop, run automatically, millions of times.

Where the analogy breaks: the engineer knows *why* they moved the bass slider. Gradient descent has no such understanding. It moves every slider slightly in whichever direction reduces the error number, with no model of what any slider means. That is precisely why the hospital model happily learned "asthma is protective".

2.1.3 Loss: putting a number on wrongness

You cannot improve what you cannot measure. A **loss function** takes the model's predictions and the true labels and returns a single number: how badly the model is doing right now. Lower is better; zero would be perfect.

Two loss functions cover most of what you will meet.

For predicting a number (a house price, a temperature), the usual choice is **mean squared error**: take each prediction, subtract the true value, square it, and average across all examples. Squaring does two things — it makes errors positive regardless of direction, and it punishes large errors disproportionately. Being wrong by £100,000 once is treated as far worse than being wrong by £10,000 ten times.

For predicting a category (spam or ham), the usual choice is **cross-entropy loss**. Rather than predicting a bare label, the model predicts a *probability*, and cross-entropy penalises confident wrongness savagely and hedged wrongness mildly.

Model says	Truth	Cross-entropy loss	Reading
99% spam	spam	very low	Confident and right — barely penalised
60% spam	spam	moderate	Right but hesitant — mild penalty
50% spam	spam	higher	No information at all
1% spam	spam	very high	Confident and wrong — punished hard

KEY TAKEAWAYS

Why cross-entropy matters far beyond this chapter

Cross-entropy over next-token probabilities **is the loss function used to train a large language model**. The model looks at a sequence, produces a probability for every token in the vocabulary, and is penalised according to the probability it assigned to the token that actually came next.

That single sentence explains an enormous amount of LLM behaviour. The model was rewarded for assigning high probability to plausible continuations. It was never directly rewarded for being *truthful*, or for saying *I don't know*. Those behaviours come later, from alignment (Chapter 7), layered on top of an objective that is fundamentally about plausibility.

Hallucination is not a bug that was introduced. It is the training objective showing through.

2.1.4 Optimisation: gradient descent without the calculus

Optimisation is the search for parameter values that minimise the loss. The workhorse algorithm is **gradient descent**, and the standard analogy is genuinely the clearest way to understand it.

Imagine a hilly landscape in thick fog. Your altitude is the loss. You want the lowest point. You cannot see anything, but you can feel the slope under your feet. So you take a small step downhill, feel again, step again. Repeat.

- The **landscape** is the loss surface — every possible combination of parameter values, and how bad each one is.
- Your **position** is the current parameter values.
- The **slope** is the *gradient*: which way is downhill, and how steeply, for each parameter.
- Your **step size** is the **learning rate**, the single most consequential setting in training.

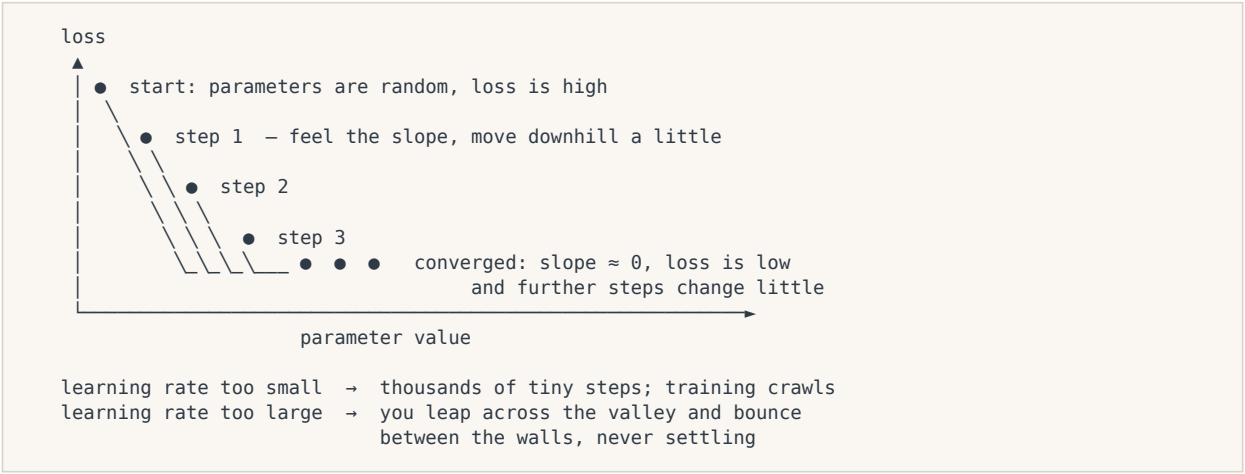


Figure 2.2 — Gradient descent as descending a foggy hillside.

That is the entire mechanism. Every neural network in this book, including the one behind Claude, was trained by repeating that loop — feel the slope, take a step — an astronomical number of times.

> NOTE
>
> **The one piece of calculus, stated and then set aside**
>
> The "slope" is formally the *partial derivative* of the loss with respect to each parameter, and computing all of them efficiently for a deep network is done by an algorithm called **backpropagation**. Chapter 3 explains backpropagation properly, still without requiring you to differentiate anything by hand. For now, "an efficient way to work out which direction is downhill for every parameter at once" is a complete and accurate summary.

2.2 The Four Paradigms of Learning

Every machine-learning system falls into one of four families, distinguished by what kind of feedback the learner receives.

Paradigm	What the data looks like	What it learns	Everyday example
Supervised	Features **with** correct labels	A mapping from features to labels	Spam filtering, credit scoring, medical triage
Unsupervised	Features, **no** labels	Structure: clusters, groupings, low-dimensional summaries	Customer segmentation, anomaly detection, topic discovery
Reinforcement	No labels; a **reward** signal after actions	A policy: what to do in each situation	Game playing, robotics, RLHF for alignment
Self-supervised	Raw unlabelled data from which labels are **derived automatically**	Rich general representations	**Pre-training every large language model**

2.2.1 Supervised learning

You have the answers, and you want the machine to reproduce them on new cases. This is by far the most common paradigm in commercial use, and it is the one worth understanding first because the other three borrow its machinery.

It splits into two sub-types by output. **Classification** predicts a category from a fixed set — spam or ham, one of eight support teams, benign or malignant. **Regression** predicts a continuous number — a price, a duration, a temperature.

The binding constraint is labels. Labels are expensive, slow, and often inconsistent between the humans producing them. A supervised project's real cost is usually the labelling effort, not the modelling.

2.2.2 Unsupervised learning

You have data and no answers, and you want the machine to find structure. **Clustering** groups similar examples together; **dimensionality reduction** compresses many features into a few that capture most of the variation; **anomaly detection** identifies examples unlike the rest.

The catch is that "structure" is not the same as "the structure you wanted". A clustering algorithm run on your customers will happily group them by time zone when you were hoping for purchasing behaviour.

Unsupervised results always require human interpretation before they mean anything.

2.2.3 Reinforcement learning

An **agent** takes **actions** in an **environment**, receives **rewards**, and learns a **policy** — a rule for choosing actions — that maximises reward over time. Nobody tells it the correct action; it only learns, eventually, that some sequences of actions turn out better than others.

The characteristic difficulty is **credit assignment**: when you win a game of chess in sixty moves, which of the sixty were good? The reward is delayed and diffuse.

> WARNING
>
> **Two different things are called "agent"**
>
> In *reinforcement learning*, an agent is the learner that adjusts a policy through reward.
>
> In the *agentic AI* sense used throughout this book and on the certification exam, an agent is a system in which a language model operates a loop — gather context, take action via tools, verify results — using a model whose parameters are entirely fixed. **No learning occurs at run time.**
>
> The words are identical; the mechanisms share almost nothing. Distractors on the exam sometimes rely on candidates conflating them. When you read "agent" in Parts 6 through 12, always assume the second meaning.

Reinforcement learning matters here for one specific reason: it is the mechanism behind **reinforcement learning from human feedback (RLHF)**, which turns a raw next-token predictor into an assistant that follows instructions and declines harmful requests. That story is Chapter 7.

2.2.4 Self-supervised learning, and why it changed everything

Here is the constraint that held the field back for decades: supervised learning needs labels, labels need humans, and humans are slow. Nobody was ever going to hand-label enough text to teach a machine language.

Self-supervised learning dissolves the problem. You take raw, unlabelled data and *manufacture* the labels from the data itself.

For text, the trick is almost embarrassingly simple. Take any sentence:

```
The capital of France is Paris.
```

Hide the last word and make it the label:

```
features:  "The capital of France is"       label:  "Paris"
```

Now slide along and do it again, at every position, in every sentence, in every document:

```
features:  "The"                            label:  "capital"
features:  "The capital"                    label:  "of"
features:  "The capital of"                 label:  "France"
features:  "The capital of France"          label:  "is"
features:  "The capital of France is"       label:  "Paris"
```

One six-word sentence has produced five labelled training examples, and **no human was involved**. Every book, article, forum post, manual and code repository ever digitised is now a labelled dataset. The bottleneck moves from human labelling effort to compute and data availability — and compute, unlike human attention, can be bought in bulk.

KEY TAKEAWAYS

This is the single most important idea in Part 1

Self-supervised next-token prediction is why large language models exist. It converted the world's text into training data without a single annotator.

Everything downstream follows from it. The model's broad general knowledge is the residue of predicting text across every domain. Its fluency is the direct target of the objective. Its confident plausibility under uncertainty is what the objective rewards. And its need for a separate alignment stage exists because "predict the next token well" and "be a helpful, honest, harmless assistant" are not the same goal.

2.3 The Training Loop

With the vocabulary in place, here is the complete procedure. Every model in this book was produced by this loop; only the scale differs.

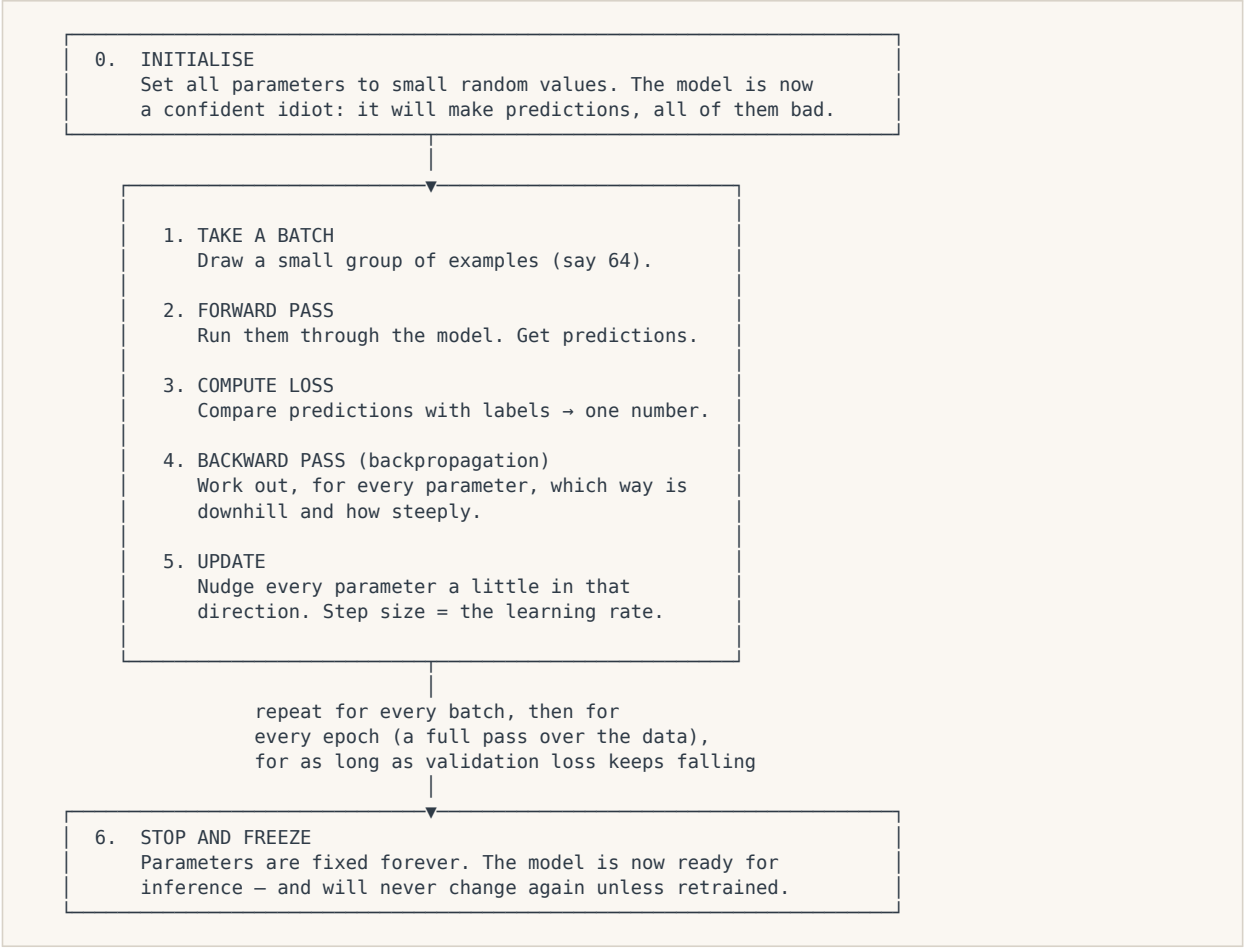


Figure 2.3 — The supervised / self-supervised training loop.

DEFINITION

Batch, epoch, learning rate

A **batch** is the group of examples processed before each parameter update. An **epoch** is one complete pass through the training data. The **learning rate** is how far parameters move at each update. These three settings are **hyperparameters** — values chosen by the engineer rather than learned by the algorithm.

Step 6 is where Chapter 1's statelessness comes from. Once training stops, the parameters are frozen. The Claude you call today is arithmetically identical to the Claude you called yesterday. Anything that looks like learning during a conversation is your application resending context, not the model changing.

2.4 Generalisation: The Whole Game

Getting a model to perform well on data it has already seen is trivial — memorise it. The entire discipline is about performing well on data it has **not** seen. That property is **generalisation**, and it is the only thing that matters.

Three regimes:

Regime	Training performance	Unseen-data performance	What went wrong
Underfitting	Poor	Poor	The model is too simple, or trained too briefly, to capture the real pattern
Good fit	Good	Good	The model captured the signal and ignored the noise
Overfitting	Excellent	Poor	The model memorised the training data, including its noise and quirks

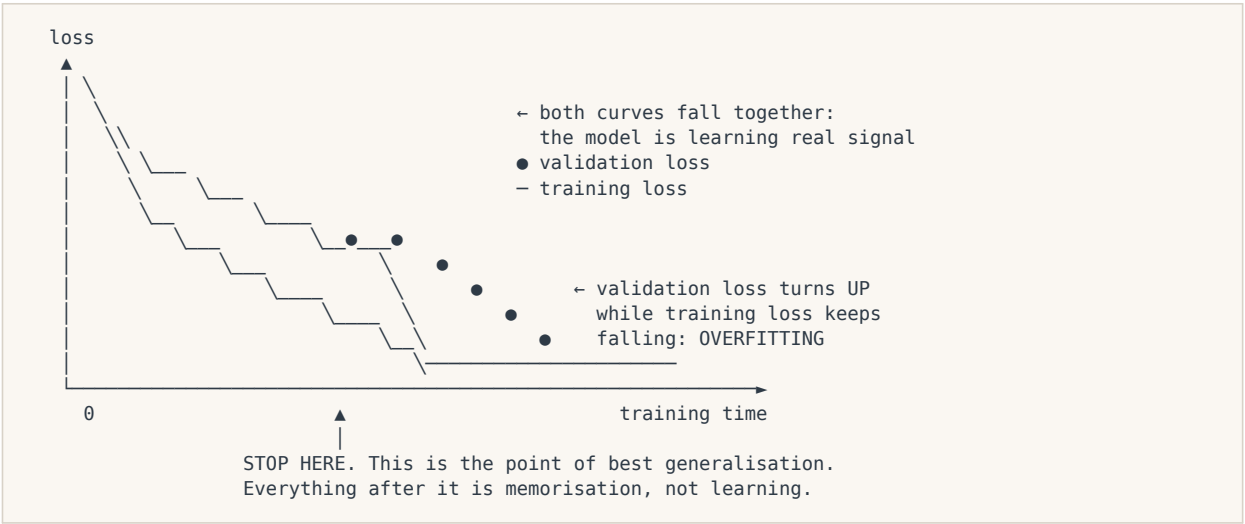


Figure 2.4 — The characteristic training and validation loss curves.

The **bias–variance trade-off** names the tension. *Bias* is error from a model too simple to represent reality — an underfitting model has high bias. *Variance* is error from a model so flexible that it chases noise — an overfitting model has high variance. Reducing one tends to increase the other, and the art is finding the point where their sum is smallest.

VISUAL ANALOGY

Three revision strategies

High bias. A student who revises by learning one rule: "the answer is usually B." Fast, consistent, and consistently mediocre on every paper.

High variance. A student who memorises the past three years' papers verbatim, answers and all. Perfect on those papers. Presented with a new one, they are lost, because they learned the specific papers rather than the subject.

Good generalisation. A student who works through past papers to extract the underlying principles, then tests themselves on questions they have never seen. This is why *validation* data must be held back and never trained on — it is the mock exam, and looking at the answers first destroys its only function.

2.5 Splitting Data, and the Four Ways to Cheat Yourself

To measure generalisation you must hold data back. The standard split:

Split	Typical share	Purpose	May the model see it?
Training	60–80%	Fit the parameters	Yes, repeatedly
Validation	10–20%	Choose hyperparameters, decide when to stop	Only for evaluation, never for parameter updates
Test	10–20%	One final, honest estimate of real-world performance	**Once**, at the very end

The test set is a one-shot instrument. The moment you look at test performance and go back to change something, it has become a second validation set and your estimate is optimistic.

COMMON MISTAKE

The four faces of data leakage

Leakage is when information that will not be available at prediction time creeps into training. It produces a model that scores brilliantly and fails in production. Four common forms:

1. **Target leakage.** A feature that is a consequence of the label. Predicting whether a customer will churn, using the field `cancellation_reason`. Available in your historical table; not available before they churn.
2. **Temporal leakage.** Randomly splitting time-series data, so the model trains on Thursday and is tested on Wednesday. Always split chronologically for anything time-ordered.
3. **Duplicate leakage.** The same or near-identical example in both training and test — very common with scraped or user-generated text. Deduplicate before splitting.
4. **Preprocessing leakage.** Computing a scaling factor, a vocabulary or an average across the *whole* dataset before splitting, so statistics from the test set influence the training pipeline. Fit every transformation on the training split alone.

The universal tell: performance that seems too good. A fraud model at 99.8% accuracy is not a triumph; it is a leak, or a class-imbalance illusion, until proven otherwise.

WARNING

Accuracy is a trap when classes are imbalanced

If 1% of transactions are fraudulent, a model that predicts "not fraud" for everything is 99% accurate and completely worthless. Use **precision** (of the things you flagged, how many were right?), **recall** (of the things you should have flagged, how many did you catch?), and their harmonic mean, **F1**. Which one you optimise is a business decision, not a technical one: a cancer screen wants recall, a spam filter that must never bin a real invoice wants precision.

2.6 Where This Sits in a Claude-Powered System

You will not train an LLM. You will, quite often, train or select a small conventional model to sit *in front of* one — and the certification tests whether you know when that is the right call.

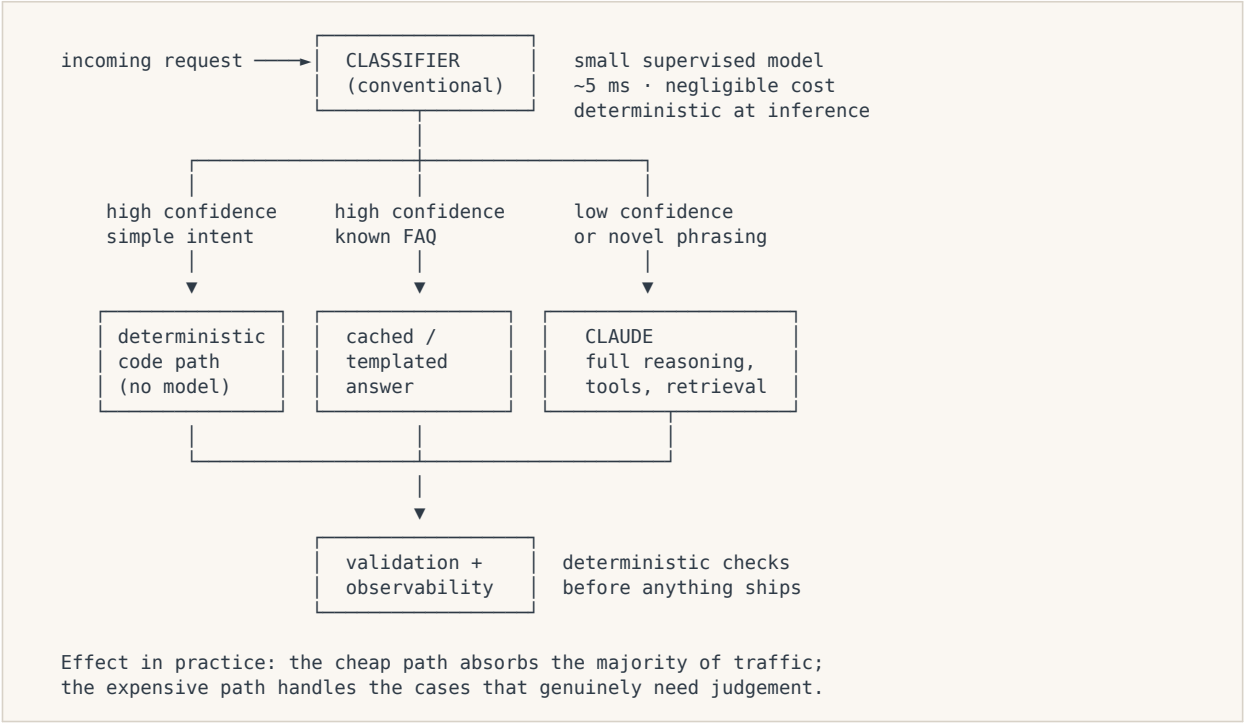


Figure 2.5 — A hybrid architecture: classical ML as a router in front of Claude.

This is one of the highest-return patterns in production AI, and it is invisible to anyone who has not internalised the material in this chapter. A classifier costing fractions of a penny per thousand calls can divert most traffic away from a model costing dollars per thousand calls, with no loss of quality on the diverted portion — because the diverted portion never needed judgement in the first place.

INDUSTRY USE CASE

Support triage at scale

A software company receives roughly 40,000 support messages a month. Analysis showed that 62% fell into eleven recurring intents with stable, correct templated answers.

The deployed design routes with a small supervised classifier trained on two years of labelled tickets. High-confidence recurring intents get a templated answer with retrieved account details. Everything else — plus anything the classifier scores below threshold — goes to Claude with full context and tools.

The reported outcome was a large reduction in model spend and a *reduction* in median latency, because the majority path no longer waits on generation. The architectural insight is the one from Chapter 1 restated: use the simplest mechanism that fully solves each case, and reserve judgement for cases that require it.

2.7 Code Walkthrough: A Classifier From Scratch

To see that there is no magic anywhere, we will build a working spam classifier in plain Python with no libraries at all, then repeat it with a standard toolkit, then compare both against a Claude API call. Reading all three side by side is the fastest way to develop judgement about which to reach for.

2.7.1 From scratch, in about sixty lines

This is a **logistic regression** classifier trained by gradient descent. Every concept from sections 2.1 to 2.3 appears in it, labelled.

```
import math
import random
# ----------------------------------------------------------------- data
# features: [n_links, n_exclamations, sender_unknown]   label: 1 = spam
DATA = [
    ([0,  0, 0], 0), ([1,  1, 0], 0), ([0,  2, 0], 0), ([2,  0, 0], 0),
    ([1,  0, 1], 0), ([0,  1, 1], 0), ([3,  1, 0], 0), ([1,  2, 0], 0),
    ([9,  7, 1], 1), ([12, 4, 1], 1), ([7,  9, 1], 1), ([15, 2, 1], 1),
    ([8,  5, 1], 1), ([11, 6, 0], 1), ([6,  8, 1], 1), ([14, 3, 1], 1),
]
N_FEATURES = 3
random.seed(7)
def sigmoid(z):
    """Squash any real number into a probability between 0 and 1."""
    if z < -60:
        return 0.0
    if z > 60:
        return 1.0
    return 1.0 / (1.0 + math.exp(-z))
def predict(weights, bias, x):
    """FORWARD PASS: features in, probability of spam out."""
    z = sum(w * xi for w, xi in zip(weights, x)) + bias
    return sigmoid(z)
def cross_entropy(weights, bias, data):
    """LOSS: one number saying how wrong the model currently is."""
    total = 0.0
    for x, y in data:
        p = min(max(predict(weights, bias, x), 1e-9), 1 - 1e-9)
        total += -(y * math.log(p) + (1 - y) * math.log(1 - p))
    return total / len(data)
def train(data, epochs=4000, learning_rate=0.05):
```

```
    # 0. INITIALISE
    weights = [random.uniform(-0.1, 0.1) for _ in range(N_FEATURES)]
    bias = 0.0
    for epoch in range(epochs):
        grad_w = [0.0] * N_FEATURES
        grad_b = 0.0
        for x, y in data:
            p = predict(weights, bias, x)      # 2. forward pass
            error = p - y                      # 4. which way is downhill
            for j in range(N_FEATURES):
                grad_w[j] += error * x[j]
            grad_b += error
        n = len(data)
        # 5. UPDATE: step downhill
        weights = [w - learning_rate * g / n for w, g in zip(weights, grad_w)]
        bias = bias - learning_rate * grad_b / n
        if epoch % 1000 == 0:
            print(f"epoch {epoch:5d}  loss {cross_entropy(weights, bias, data):.4f}")
    return weights, bias
weights, bias = train(DATA)
print("\nlearned weights:", [round(w, 3) for w in weights])
print("learned bias   :", round(bias, 3))
for x in ([0, 0, 0], [10, 6, 1], [2, 1, 1]):
    p = predict(weights, bias, x)
    print(f"{x} -> P(spam) = {p:.3f} -> {'SPAM' if p > 0.5 else 'HAM'}")
```

What to notice, in order:

- `sigmoid` turns an unbounded score into a probability. Without it the model would output numbers like −4.7 or 12.3, which cannot be interpreted as confidence.
- `predict` is the forward pass and the model itself: a weighted sum plus a bias, squashed. Three parameters and one bias. Claude has the same structure repeated across billions of parameters and many layers.
- `cross_entropy` is the loss from section 2.1.3, implemented literally. Confident wrongness produces a large logarithm; hedged wrongness a small one.
- `error = p - y` is the gradient for this model, and it is worth pausing on: for logistic regression the direction of steepest descent turns out to be simply *how far off you were*, multiplied by the feature. Backpropagation in Chapter 3 is the generalisation of this to many layers.
- **The update line** is step 5 of Figure 2.3. Move each parameter a little way against the gradient. `learning_rate` is the step size.

Run it and the loss falls steadily. Inspect the learned weights and you will find `n_links` carries the most weight — which the model discovered on its own, because nobody told it that links matter.

2.7.2 The same thing with a standard toolkit

In practice nobody hand-writes gradient descent. The scikit-learn version is four lines, and it is worth seeing precisely how much the previous section demystified.

```
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report
X = [x for x, _ in DATA]
y = [label for _, label in DATA]
# Split BEFORE anything else. This is where leakage is prevented.
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y
)
model = LogisticRegression().fit(X_train, y_train)
print(classification_report(y_test, model.predict(X_test),
                            target_names=["ham", "spam"]))
print("coefficients:", model.coef_, "intercept:", model.intercept_)
```

`stratify=y` preserves the class balance across the split — with imbalanced data, omitting it can produce a test set containing no positive examples at all, and a meaningless score.

2.7.3 The same task with Claude

```
import json
import os
from anthropic import Anthropic
client = Anthropic(api_key=os.environ["ANTHROPIC_API_KEY"])
SYSTEM = """You classify emails as spam or ham.
Respond with JSON only, no prose:
{"label": "spam" | "ham", "confidence": 0.0-1.0, "reason": "<8 words"}"""
def classify(subject: str, body: str) -> dict:
    r = client.messages.create(
        model="claude-haiku-4-5",
        max_tokens=100,
        system=SYSTEM,
        messages=[{
            "role": "user",
            "content": f"Subject: {subject}\n\n{body}",
        }],
    )
    if r.stop_reason == "max_tokens":
        raise ValueError("truncated output")
    text = "".join(b.text for b in r.content if b.type == "text")
    return json.loads(text)
print(classify("URGENT: claim your prize",
               "Click here now!!! Limited time offer!!!"))
```

2.7.4 Choosing between them

Dimension	From scratch / scikit-learn	Claude API
Training data required	Yes — hundreds to thousands of labelled examples	None
Time to first working version	Days to weeks (mostly labelling)	Minutes
Latency per prediction	Under a millisecond	Hundreds of milliseconds
Cost per million predictions	Effectively the cost of a CPU	Meaningful; scales with tokens
Determinism at inference	Yes	No
Handles phrasings never seen before	Poorly	Well
Explains its reasoning	Only via coefficients	Yes, in natural language

Dimension	From scratch / scikit-learn	Claude API
Adapting to a new category	Retrain and redeploy	Edit one line of the prompt
Auditability	Complete	Requires deliberate logging

BEST PRACTICE

The honest decision rule

Use a **conventional model** when you have labelled data, high volume, a stable and small output space, and latency or cost pressure.

Use **Claude** when you have no labelled data, the input is open-ended, categories change, you need a justification alongside the answer, or you are still discovering what the categories should be.

Use **both** when volume is high but the tail is messy — which describes most real systems. Prototype with Claude to establish that the task is solvable and to *generate labels cheaply*; then, if volume justifies it, distil a small classifier for the common path and keep Claude for the tail. That is the architecture in Figure 2.5, and "use the LLM to bootstrap the labels for the classifier" is one of the most useful moves in the field.

2.8 Hands-on Labs

Lab 2.1 — Watch a model learn. Run the from-scratch classifier and print the loss every 100 epochs instead of every 1000. Plot or eyeball the curve. *Success criterion:* you can point to the region where learning was fastest and explain why the curve flattens.

Lab 2.2 — Break it with the learning rate. Re-run with `learning_rate=5.0` and with `learning_rate=0.0001`. *Success criterion:* you can describe both failure modes from your own output and connect each to Figure 2.2.

Lab 2.3 — Manufacture overfitting. Add nine random, meaningless features to each example. Retrain. Compare training loss with performance on three examples you invent by hand. *Success criterion:* training loss improves while real-world behaviour degrades, and you can explain why.

Lab 2.4 — Build a leak, then find it. Add a feature `was_reported_by_user` that is 1 for every spam example and 0 for every ham example. Train, observe near-perfect performance, then explain why the model is useless in production. *Success criterion:* you can name which of the four leakage types this is.

Lab 2.5 — Bootstrap labels with Claude. Take fifty unlabelled sentences of your own. Use the Claude classifier from 2.7.3 to label them, then train the scikit-learn model on those labels and evaluate on ten you labelled by hand. *Success criterion:* a number for how much accuracy you lost by distilling, and an opinion on whether the trade was worth it.

2.9 Lab Solutions

Solution 2.1. Loss falls steeply at first, then flattens. Early on, parameters are far from good values and the gradient is large, so each step covers a lot of ground. Near the minimum the slope is shallow and each step achieves little. This is exactly why "train for longer" stops helping — and why the stopping criterion should be *validation loss no longer improving*, not a fixed epoch count.

Solution 2.2. At `5.0` the loss oscillates or explodes to `nan`: the step overshoots the valley and lands higher up the opposite wall, repeatedly. At `0.0001` the loss barely moves in 4,000 epochs: correct direction, useless pace. Both are the same defect — a step size mismatched to the shape of the loss surface. In modern practice this is handled by adaptive optimisers and learning-rate schedules, but the failure modes are unchanged.

```
for lr in (5.0, 0.05, 0.0001):
    print(f"\n--- learning_rate = {lr} ---")
    w, b = train(DATA, epochs=2000, learning_rate=lr)
    print("final loss:", round(cross_entropy(w, b, DATA), 5))
```

Solution 2.3. With sixteen examples and twelve features the model has more than enough freedom to memorise. It finds spurious combinations of noise features that happen to separate the training rows, drives training loss towards zero, and predicts nonsense on anything new. This is the *curse of dimensionality* meeting a small dataset. The mitigations are more data, fewer features, regularisation, and — always — a held-out set that would have exposed it immediately.

```
import random
NOISY = [(x + [random.random() for _ in range(9)], y) for x, y in DATA]
```

Solution 2.4. Target leakage. `was_reported_by_user` is a *consequence* of the message being spam, not a property available at the moment you must decide. In production the field is empty for every new message. The general test is worth memorising: **for each feature, ask whether its value is knowable at the exact instant the prediction must be made.** If not, delete it.

Solution 2.5. Expect to lose some accuracy — typically a few percentage points on clean tasks, more where phrasing is varied. The trade is usually worth it when volume is high and the categories are stable, because you exchange a per-call cost for a fixed one. It is a bad trade when the categories are still changing, because every change means relabelling and retraining. This is precisely the judgement the exam probes.

2.10 Best Practices

BEST PRACTICE

Eight rules that prevent most machine-learning disasters

1. **Split before you look.** Hold out test data before any exploration, transformation or feature engineering.
2. **Establish a trivial baseline first.** Always-predict-the-majority-class, or a keyword rule. If your model cannot beat it convincingly, you have learned something important very cheaply.
3. **Choose the metric before you see results.** Precision, recall or F1 — decided by the cost of each error type, not by which number looks best afterwards.
4. **Distrust excellent scores.** Investigate before celebrating. Suspiciously good is a symptom, not an achievement.
5. **Audit what the model actually keyed on.** Inspect coefficients, importances or attributions. This is how the asthma rule was caught.
6. **Version data as rigorously as code.** A model is a function of its data. An unversioned dataset makes a result unreproducible.
7. **Monitor for drift after deployment.** The world moves; a model frozen at training time slowly stops matching it. Track input distributions and outcome rates, not just uptime.
8. **Write down the failure cost.** What happens when this model is wrong, and who absorbs it? That answer determines your threshold, your human-review policy and whether the system should ship at all.

2.11 Common Mistakes

COMMON MISTAKE

Evaluating on the training data

Symptom: superb metrics, disappointing production. **Cause:** measuring memorisation rather than generalisation. **Fix:** a strict train/validation/test discipline, with the test set touched once. **Detection:** if the number came from data the model was fitted on, it is not evidence.

COMMON MISTAKE

Reporting accuracy on imbalanced classes

Symptom: 99% accuracy on a rare-event problem. **Cause:** the majority class dominates the metric. **Fix:** precision, recall, F1, and the confusion matrix. **Detection:** compare against always predicting the majority class — if the gap is small, the number is meaningless.

COMMON MISTAKE

Tuning against the test set

Symptom: the model looks better each time you adjust it, then underperforms in production. **Cause:** repeated exposure turns the test set into a validation set and you have optimised against it. **Fix:** tune on validation only; reserve test for a single final measurement.

COMMON MISTAKE

Assuming more data always helps

Symptom: cost and training time grow, performance does not. **Cause:** the model was underfitting, or the additional data duplicates what was already there. **Fix:** diagnose first. High training *and* validation loss means underfitting — more capacity or better features, not more rows.

COMMON MISTAKE

Believing that fine-tuning is the answer to everything

Symptom: a plan to fine-tune before anyone has tried prompting properly. **Cause:** "training" sounds more serious than "prompting". **Fix:** exhaust prompt engineering, retrieval and tool use first. Fine-tuning is expensive, freezes behaviour, needs curated data and must be repeated for every model upgrade. This appears on the exam as a distractor with reliable frequency.

COMMON MISTAKE

Conflating the two meanings of "agent"

Symptom: a design document claiming the agent "learns from user corrections over time" with no training pipeline anywhere. **Cause:** importing reinforcement-learning intuitions into an agentic-AI context. **Fix:** if you want persistence across sessions, build storage and retrieval. The model does not change.

2.12 Performance Considerations

Training cost and inference cost are unrelated. A model can take a week to train and a microsecond to run. When you compare a small classifier with an LLM, compare the *inference* profiles — that is what recurs on every request. Training is a one-off capital cost.

A small model in front of a large one is usually the best latency optimisation available. A classifier answering in five milliseconds for 60% of traffic changes the median response time of the whole system far more than any prompt tuning could.

Feature computation is often the real bottleneck. If a feature requires a database round trip, the model's own speed is irrelevant. Profile the whole path.

Batch what you can. Both conventional models and the Claude API are more efficient per item when work is batched. For anything not user-facing — nightly classification, backfills, bulk enrichment — batch processing changes the economics substantially. Chapter 30 covers the Claude side.

2.13 Security Considerations

Training data is an attack surface. If an adversary can influence what goes into your dataset — through user-generated content, public scraping or a compromised pipeline — they can shape what the model learns. This is **data poisoning**, and it is a supply-chain problem: know the provenance of every dataset you train on.

Models leak their training data. A model can reproduce distinctive strings it was trained on. Never train a model on personal or confidential data that its users are not entitled to see. Redaction belongs *before* training, not after.

Learned proxies encode discrimination. A model given no protected attribute can still learn a proxy for one — postcode standing in for ethnicity, employment gap standing in for parenthood. In regulated domains this is a legal exposure, not merely an ethical one. Audit for it deliberately; it will not surface on its own.

Predictions are not decisions. A model output is evidence. Wrapping it in a policy — thresholds, human review, appeal routes, an audit trail — is what turns it into a defensible decision. That wrapper is architecture, and it is your responsibility.

2.14 Real Production Examples

Fraud detection. A supervised classifier scores every transaction in single-digit milliseconds; a rules layer handles hard regulatory constraints; borderline cases go to a human queue. Note the shape — model, rules and human, in one system. Nobody serious builds fraud detection out of any one of the three alone.

Recommendation. Unsupervised structure (which items cluster together) combined with supervised signals (what this user clicked). The interesting engineering problem is the feedback loop: recommendations change behaviour, behaviour becomes tomorrow's training data, and the system can converge on a narrow, self-reinforcing slice of the catalogue.

Content moderation. A fast conventional classifier handles unambiguous cases at volume; an LLM handles context-dependent judgement calls where sarcasm, quotation or reclaimed language matter; a human handles appeals. The routing thresholds are the product decision, and they are tuned on validation data with an explicit cost model for each error type.

Demand forecasting. Regression over time-series features. The instructive failure is temporal leakage: teams who split randomly report excellent accuracy in development and are surprised in production, because the model was quietly allowed to see the future.

2.15 Mini Project: The Hybrid Router

Goal. Build the architecture in Figure 2.5: a small trained classifier that routes support messages, escalating to Claude only when it is not confident. Measure how much traffic you divert, and what it costs you in quality.

Architecture. Incoming message → TF-IDF vectoriser → logistic-regression classifier → confidence check → templated answer *or* Claude → validation → response, with every decision logged.

Folder structure.

```
hybrid-router/
├── README.md
├── requirements.txt
├── data/
```

```
│   ├── labelled.csv          # message,intent
│   └── holdout.csv           # never touched until the end
├── router/
│   ├── __init__.py
│   ├── train.py              # fit and persist the classifier
│   ├── classify.py           # load model, predict with confidence
│   ├── escalate.py           # Claude fallback
│   └── serve.py              # end-to-end routing + logging
└── tests/
    └── test_router.py
```

Training the router.

```
# router/train.py
import joblib
import pandas as pd
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import Pipeline
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report
df = pd.read_csv("data/labelled.csv")
X_train, X_val, y_train, y_val = train_test_split(
    df["message"], df["intent"],
    test_size=0.2, random_state=42, stratify=df["intent"],
)
pipe = Pipeline([
    ("tfidf", TfidfVectorizer(ngram_range=(1, 2), min_df=2)),
    ("clf", LogisticRegression(max_iter=1000, class_weight="balanced")),
])
pipe.fit(X_train, y_train)
print(classification_report(y_val, pipe.predict(X_val)))
joblib.dump(pipe, "router/model.joblib")
```

Note `class_weight="balanced"`: support intents are never evenly distributed, and without it the classifier will simply learn to predict your most common intent.

Routing with a confidence threshold.

```
# router/classify.py
import joblib
_pipe = joblib.load("router/model.joblib")
def classify(message: str):
    """Return (intent, confidence)."""
    probs = _pipe.predict_proba([message])[0]
    idx = probs.argmax()
    return _pipe.classes_[idx], float(probs[idx])
```

```
# router/serve.py
import time
from .classify import classify
from .escalate import ask_claude
THRESHOLD = 0.80
TEMPLATES = {
    "reset_password": "You can reset your password at /account/reset ...",
    "refund_status": "Refunds are processed within 5 working days ...",
    "shipping_delay": "We're sorry for the delay. Your parcel ...",
}
```

```
def handle(message: str) -> dict:
    started = time.perf_counter()
    intent, confidence = classify(message)
    if confidence >= THRESHOLD and intent in TEMPLATES:
        answer, path = TEMPLATES[intent], "template"
    else:
        answer, path = ask_claude(message), "claude"
    return {
        "answer": answer,
        "path": path,                       # log this — it is your dashboard
        "intent": intent,
        "confidence": round(confidence, 3),
        "latency_ms": round((time.perf_counter() - started) * 1000, 1),
    }
```

```
# router/escalate.py
import os
from anthropic import Anthropic
client = Anthropic(api_key=os.environ["ANTHROPIC_API_KEY"])
SYSTEM = (
    "You are a support agent for an online retailer. Answer in under 120 "
    "words, in plain English. If you do not have enough information, say "
    "exactly what you need. Never invent order numbers, dates or policies."
)
def ask_claude(message: str) -> str:
    r = client.messages.create(
        model="claude-opus-5",
        max_tokens=400,
        system=SYSTEM,
        messages=[{"role": "user", "content": message}],
    )
    if r.stop_reason == "max_tokens":
        return ("I need a moment to give you a complete answer \u2014 "
                "let me connect you to a colleague.")
    return "".join(b.text for b in r.content if b.type == "text")
```

Testing.

```
# tests/test_router.py
from router.serve import handle, THRESHOLD
from router.classify import classify
def test_confident_intent_uses_template():
    result = handle("How do I reset my password?")
    assert result["path"] == "template"
    assert result["confidence"] >= THRESHOLD
def test_novel_message_escalates():
    result = handle("My dog ate the delivery note, what now?")
    assert result["path"] == "claude"
def test_confidence_is_a_probability():
    _, c = classify("where is my refund")
    assert 0.0 <= c <= 1.0
```

Deployment. Wrap `handle()` in a small HTTP service. Persist `model.joblib` as a versioned build artefact — never train at start-up. Emit one structured log line per request containing intent, confidence, path, latency and token usage. Put the Claude key in a secrets manager.

Evaluation. Three numbers, measured on the holdout file you have not touched:

1. **Deflection rate.** What proportion took the template path? This is your cost saving.
2. **Template accuracy.** Of the deflected messages, how many received the *right* template? Sample fifty and check by hand. This is your quality risk, and it is the number that decides your threshold.
3. **Latency split.** Median for each path. The gap is what your users actually feel.

Then sweep `THRESHOLD` from 0.5 to 0.95 and plot deflection against template accuracy. The resulting curve *is* the business decision, made visible. Nothing in this book will teach you more about production trade-offs than staring at that curve for ten minutes.

2.16 Certification Notes

Chapter 2 is foundational rather than directly examinable, but four of its ideas are load-bearing for scenarios across all five domains.

Concept	Where it resurfaces on the exam
Generalisation and held-out evaluation	Domain 4: designing prompt evaluation sets, and why testing a prompt on the examples used to write it proves nothing
Overfitting	Domain 4: few-shot examples so specific that the prompt fails on real inputs
The frozen-parameters fact	Domains 1 and 5: subagent context isolation, conversation compaction, and why nothing persists on its own
Simplest sufficient mechanism	Domain 1: agent vs. workflow vs. single call — the single most common question shape
Confidence thresholds and escalation	Domain 1: human-in-the-loop design; Domain 5: reliability patterns

CERTIFICATION NOTE

Exam tips carried forward from this chapter

- Any option asserting that the model "learns from user interactions" or "improves over time" without describing a training or retrieval pipeline is wrong. Parameters are frozen.
- Fine-tuning is rarely the correct exam answer. Prompting, retrieval and tool use come first; fine-tuning appears mainly as an expensive distractor.
- When a scenario mentions high volume plus a stable, narrow task, expect the correct answer to involve a cheaper model, a cache, a classifier or a deterministic path — not a more capable model.
- Evaluation questions reward held-out data and explicit metrics. Options describing evaluation on the same examples used to build the prompt are wrong for the reasons in section 2.5.

2.17 Practice Questions

1. What distinguishes supervised from unsupervised learning? (a) Supervised uses neural networks; unsupervised does not. (b) Supervised training data carries correct labels; unsupervised data does not. (c) Supervised runs on GPUs; unsupervised on CPUs. (d) Supervised is for text; unsupervised is for images.

2. A model achieves 99% training accuracy and 61% on held-out data. What is happening? (a) Underfitting (b) Overfitting (c) Data leakage (d) Class imbalance

3. What is a loss function? (a) The money lost when a model is wrong. (b) A single number quantifying how wrong the model's current predictions are. (c) The proportion of examples discarded during training. (d) The rate at which parameters decay.

4. Which best describes gradient descent? (a) Randomly sampling parameter values and keeping the best. (b) Repeatedly nudging parameters in the direction that reduces loss. (c) Adding layers until performance improves. (d) Reducing the dataset until training becomes fast.

5. Why was self-supervised learning transformative for language models? (a) It removed the need for computation. (b) It generates training labels automatically from raw text, removing the human labelling bottleneck. (c) It guarantees factual accuracy. (d) It allows models to keep learning after deployment.

6. Predicting churn using a feature named `cancellation_reason` is an example of: (a) Feature engineering (b) Target leakage (c) Regularisation (d) Class balancing

7. A fraud model reports 99.4% accuracy where 0.6% of transactions are fraudulent. What should you conclude? (a) The model is excellent. (b) Accuracy is uninformative here; check precision and recall. (c) The training set was too small. (d) The learning rate was too high.

8. What is the purpose of the validation set? (a) To fit the parameters. (b) To tune hyperparameters and decide when to stop training. (c) To provide the final unbiased performance estimate. (d) To increase the size of the training data.

9. During a conversation, a user corrects Claude. What happens to the model? (a) Its parameters update to reflect the correction. (b) Nothing; parameters are frozen. The correction only exists in the context you resend. (c) A fine-tuning job is queued automatically. (d) The correction is stored in the model's long-term memory.

10. A learning rate set far too high causes: (a) Extremely slow but steady convergence. (b) Loss that oscillates or diverges because steps overshoot the minimum. (c) Guaranteed overfitting. (d) Automatic early stopping.

11. Which is the strongest argument for a small classifier in front of Claude? (a) Classifiers are more accurate than LLMs on every task. (b) A high-volume stream contains a large share of repetitive, well-defined cases that do not need judgement. (c) LLMs cannot classify. (d) It removes the need for evaluation.

12. Underfitting is best diagnosed by: (a) High training loss and high validation loss. (b) Low training loss and high validation loss. (c) Low training loss and low validation loss. (d) High training loss and low validation loss.

13. Which is *not* a form of data leakage? (a) Random splitting of time-series data. (b) Computing a scaler over the whole dataset before splitting. (c) Duplicate records appearing in train and test. (d) Using class weights to handle imbalance.

14. Cross-entropy loss penalises which case most heavily? (a) 55% confidence, correct (b) 95% confidence, correct (c) 2% confidence on the true class (d) 50% confidence, correct

15. A team proposes fine-tuning a model to make it follow a new output format. What should you suggest first? (a) Fine-tune immediately; formatting requires it. (b) Try a system prompt with an explicit schema and few-shot examples, plus deterministic validation. (c) Train a model from scratch. (d) Increase `max_tokens`.

16. In reinforcement learning, the credit assignment problem refers to: (a) Attributing cloud costs across teams. (b) Determining which earlier actions were responsible for a delayed reward. (c) Assigning labels to unlabelled examples. (d) Splitting data between train and test.

17. Which statement about parameters is correct? (a) They are chosen by the engineer before training. (b) They are learned during training and frozen at inference. (c) They change on every API request. (d) They are the same as hyperparameters.

18. A support classifier reports 0.42 confidence on a message. Given a 0.80 threshold, the correct action is: (a) Return the top intent's template anyway. (b) Escalate to Claude or a human. (c) Retrain the classifier on this message immediately. (d) Return an empty response.

19. You are asked to evaluate a prompt. Which approach is sound? (a) Test it on the same examples you used while writing it. (b) Build a held-out set of realistic inputs with expected properties, and score against it. (c) Ask the model whether the prompt is good. (d) Ship it and monitor complaints.

20. Why can a model trained without any protected attribute still discriminate? (a) It cannot; excluding the attribute is sufficient. (b) It can learn correlated proxies that stand in for the excluded attribute. (c) Only deep networks can do this. (d) Only if the dataset is small.

2.18 Answers and Explanations

1 — (b). The presence or absence of labels is the defining distinction. Every other option confuses an implementation detail with the paradigm.

2 — (b). A large gap between training and held-out performance is the textbook overfitting signature. Leakage (c) usually inflates *both*, which is why it is the more dangerous failure.

3 — (b). A single scalar measuring current wrongness, which optimisation then minimises.

4 — (b). Iterative descent along the gradient. (a) describes random search, which is hopeless at scale.

5 — (b). Labels are derived from the data itself, so all digitised text becomes training data. (d) is a common and important misconception — models do not learn after deployment.

6 — (b). The feature is a consequence of the outcome and is unavailable at prediction time.

7 — (b). With 0.6% positives, predicting "not fraud" always yields 99.4%. The metric carries no information about the thing you care about.

8 — (b). Validation guides hyperparameters and early stopping. (c) describes the test set, which must be used once.

9 — (b). Parameters are frozen at inference. This links directly to Chapter 1's statelessness and to subagent context isolation later.

10 — (b). Overshooting produces oscillation or divergence. (a) describes the opposite error.

11 — (b). The argument is about traffic shape and cost, not about one technology being universally superior.

12 — (a). Poor on both means the model never captured the pattern. (b) is overfitting.

13 — (d). Class weighting is a legitimate technique for imbalance. The other three are canonical leakage patterns.

14 — (c). Confident wrongness attracts the largest penalty — this is the property that makes cross-entropy suitable for calibrating probability estimates.

15 — (b). Prompting with an explicit schema plus deterministic validation is cheaper, faster to iterate and easier to change. Fine-tuning for formatting is the classic over-engineered distractor.

16 — (b). Determining which of many earlier actions produced a delayed reward.

17 — (b). Learned during training, frozen at inference. Hyperparameters (learning rate, batch size) are the ones set by the engineer.

18 — (b). Low confidence is precisely the signal the threshold exists to act on. (c) misunderstands the architecture — there is no online training.

19 — (b). Held-out evaluation is the direct application of section 2.5 to prompt engineering, and it is how Domain 4 questions are framed.

20 — (b). Correlated proxies. This is why fairness requires deliberate auditing rather than simply dropping a column.

2.19 Ten Practical Exercises

1. Re-implement the from-scratch classifier for a regression target using mean squared error instead of cross-entropy.
2. Add L2 regularisation to the from-scratch trainer and show empirically that it reduces overfitting on the noisy-feature dataset from Lab 2.3.
3. Take a public dataset, split it three ways correctly, and write down before you start which metric you will optimise and why.
4. Deliberately introduce each of the four leakage types into a pipeline, and write the check that would catch each one.
5. Build a confusion matrix by hand from twenty predictions and compute precision, recall and F1 without a library.
6. Use Claude to label 200 unlabelled examples, then train a classifier on them and quantify the distillation loss.
7. Sweep the confidence threshold in the mini project and plot deflection rate against template accuracy.
8. Write a one-page memo explaining to a non-technical stakeholder why 99% accuracy on their problem might be worthless.
9. Take an existing model in your organisation and audit which three features carry the most weight. Ask whether each is causal or merely correlated.

10. Design a drift monitor: what would you measure weekly to notice that a deployed classifier has quietly stopped matching reality?

2.20 Five Challenge Labs

Challenge 1 — The leakage detector. Write a tool that takes a dataframe and a target column and flags suspicious features automatically: near-perfect single-feature separation, high correlation with the target, and columns whose values are missing for all recent rows.

Challenge 2 — The distillation pipeline. Automate the loop in Lab 2.5: Claude labels, classifier trains, both are evaluated on a fixed holdout, and the system reports whether the classifier is now good enough to take over the common path.

Challenge 3 — The threshold optimiser. Extend the mini project so the confidence threshold is chosen automatically to maximise deflection subject to a user-specified minimum template accuracy. Report the chosen threshold and the trade it implies.

Challenge 4 — The drift alarm. Log input feature distributions and predicted class frequencies daily. Alert when either shifts beyond a statistical threshold. Then deliberately shift your inputs and confirm the alarm fires.

Challenge 5 — The honest evaluation. Take a model you or a colleague has built and try to break its evaluation. Look for leakage, for a test set that has been peeked at, for a metric chosen after the fact, and for a missing baseline. Write up what you find. This is the single most valuable skill in the chapter.

2.21 Interview Questions

1. Explain overfitting to someone with no technical background, then explain how you detect it.
2. What is the difference between a parameter and a hyperparameter?
3. Why is accuracy a poor metric for rare-event problems, and what would you use instead?
4. Describe gradient descent without using the word "derivative".
5. What is self-supervised learning, and why did it matter more than any architectural advance for language models?
6. Give three concrete examples of data leakage and the check that catches each.
7. When would you choose a small conventional classifier over an LLM, and when the reverse?
8. A stakeholder wants to fine-tune a model. What questions do you ask before agreeing?
9. Explain the bias–variance trade-off using an example from your own work.
10. How would you evaluate a prompt, and how does that differ from evaluating a trained model?

2.22 Summary

Machine learning is the discipline of deriving behaviour from data rather than writing it by hand. A model is a function containing adjustable parameters; a loss function scores how wrong its current predictions are; gradient descent repeatedly nudges the parameters downhill on that loss; and training is

that loop, repeated until validation performance stops improving. Then the parameters freeze, permanently.

Four paradigms cover the field. Supervised learning maps labelled features to answers. Unsupervised learning finds structure in unlabelled data. Reinforcement learning derives a policy from delayed rewards. Self-supervised learning manufactures its own labels from raw data, and it is the reason large language models exist at all: by turning every digitised sentence into training examples, it removed the human labelling bottleneck that had constrained the field for fifty years.

The only thing that matters is generalisation — performance on data never seen. That requires held-out evaluation, honest metrics chosen in advance, and vigilance against the four kinds of leakage, because a model that scores brilliantly by cheating is worse than no model at all: it carries false confidence into production.

Finally, the practical lesson that recurs for the rest of the book: conventional machine learning has not been superseded by language models. It sits alongside them. A small classifier routing traffic in front of Claude is frequently the difference between a system that is affordable and one that is not — and recognising when to reach for it is exactly the architectural judgement the certification measures.

KEY TAKEAWAYS

1. **Learning means adjusting parameters to reduce a loss.** Nothing more mysterious than that is happening.
2. **Cross-entropy over next tokens is how LLMs are trained** — which is why fluency and plausibility, not truth, are the native objective.
3. **Self-supervision removed the labelling bottleneck.** It is the single ingredient that made LLMs possible.
4. **Parameters freeze after training.** All apparent memory in a Claude system is architecture you built.
5. **Generalisation is the only goal.** Training performance is not evidence of anything.
6. **Overfitting is memorisation; underfitting is inadequacy.** Diagnose from the gap between training and validation loss.
7. **Leakage is the most dangerous failure** because it looks like success.
8. **Accuracy lies under class imbalance.** Choose the metric before you see the numbers.
9. **Fine-tuning is rarely the first answer.** Prompting, retrieval and tools come first.
10. **Hybrid beats pure.** A cheap classifier in front of an expensive model is one of the highest-return patterns in production AI.

2.23 Glossary

Term	Definition
Backpropagation	The algorithm that efficiently computes, for every parameter, the direction that reduces loss.
Batch	The group of examples processed before each parameter update.
Bias (statistical)	Error arising from a model too simple to represent the underlying pattern.

Term	Definition
Bias–variance trade-off	The tension between a model too simple to fit and one so flexible it fits noise.
Classification	Supervised learning where the output is a category from a fixed set.
Cross-entropy	The loss function used for probabilistic classification, and for next-token prediction in LLMs.
Data leakage	Information available in training but not at prediction time, producing falsely good results.
Dataset	A collection of examples used for training or evaluation.
Epoch	One complete pass through the training data.
Feature	A measurable input property supplied to a model.
Generalisation	Performance on data never seen during training. The only measure that matters.
Gradient descent	Iteratively adjusting parameters in the direction that most reduces loss.
Hyperparameter	A setting chosen by the engineer rather than learned — learning rate, batch size, model size.
Label	The correct output for an example in supervised learning.
Learning rate	The size of each parameter update step.
Loss function	A scalar measure of how wrong the model's current predictions are.
Mean squared error	The standard loss for regression: average of squared differences.
Overfitting	Memorising training data, including its noise, at the expense of generalisation.
Precision	Of the examples flagged positive, the proportion that truly were.
Recall	Of the examples that were truly positive, the proportion that were flagged.
Regression	Supervised learning where the output is a continuous number.
Regularisation	Techniques that constrain model flexibility to reduce overfitting.
Reinforcement learning	Learning a policy from delayed reward signals.
Self-supervised learning	Deriving labels automatically from raw data; the basis of LLM pre-training.
Supervised learning	Learning a mapping from labelled examples.
Test set	Data held back for a single, final, unbiased performance estimate.
Underfitting	Failing to capture the real pattern, producing poor performance everywhere.
Unsupervised learning	Finding structure in data without labels.
Validation set	Data used to tune hyperparameters and decide when to stop training.

2.24 Further Reading

Primary and canonical sources.

- Claude Platform documentation, *Prompt engineering overview* — the evaluation discipline in section 2.5 applied directly to prompts.
- Claude Platform documentation, *Batch processing* — the production economics of bulk classification.
- Caruana et al. (2015), *Intelligible Models for HealthCare* — the source of the asthma example that opens this chapter. Short, readable, and permanently useful.
- Hastie, Tibshirani and Friedman, *The Elements of Statistical Learning* — the standard reference for the mathematics deliberately set aside here.
- Sculley et al. (2015), *Hidden Technical Debt in Machine Learning Systems* — why the model is the smallest part of the system, argued from production experience.

Where to go next in this book.

Chapter 3 builds a neural network from the logistic-regression model in section 2.7, one layer at a time, and explains backpropagation properly. Chapter 4 then assembles those layers into a transformer. If you are following Route B or C from *How to Use This Book*, you may skip directly to Chapter 14 — but return here before attempting Domain 4, because prompt evaluation is this chapter's material under another name.